Real-Time Schedulers for STM32: Performance Evaluation of EDF over SuperLoop

Vladimir Toporkov, Artem Burmyakov
Innopolis University

I. Introduction

STM32 microcontrollers are used in embedded applications, including environmental [2] and equipment condition monitoring [3]. As workload increases, the scheduling approach can affect task response time and the ability to meet deadlines.

Two approaches to task scheduling are SuperLoop [4] and Earliest Deadline First (EDF) [5]. SuperLoop executes tasks in a fixed polling order without preemption, while EDF selects the ready job with the earliest absolute deadline. In chunked EDF, scheduling decisions are made between execution chunks rather than immediately upon each job release.

Under SuperLoop, a newly released job may wait until the running job completes. Chunked EDF can switch to another ready job at a chunk boundary. These differences motivate an experimental comparison of responsiveness and scheduling overhead on STM32.

A. Contribution

We compare SuperLoop and chunked EDF for periodic tasks with implicit deadlines on an STM32F767 microcontroller. We evaluate the effects of task-set utilization on response time, deadline misses, skipped releases, and measured task-selection overhead. We additionally investigate how maximum chunk duration affects these metrics for chunked EDF. Both scheduler implementations are publicly available.

II. Background on STM32

Fig. 1 illustrates the sensor platform considered in this work, based on an STM32F767 uniprocessor. Table I summarizes the functions and interfaces of its four sensors.

TABLE I
Sensor functions and interfaces.

Sensor	Purpose	Interface
DHT11 [6]	Temperature and humidity	Single-wire digital (GPIO)
HC-SR04 [7]	Distance	Trigger/echo (GPIO)
OV2640 [8]	Image acquisition	Parallel data, SCCB
SPW2430 [9]	Signal acquisition	Analog output (ADC)

General-purpose input/output (GPIO) pins carry digital signals, while an analog-to-digital converter (ADC) converts an analog signal into digital values. The Serial Camera Control Bus (SCCB) configures the camera.

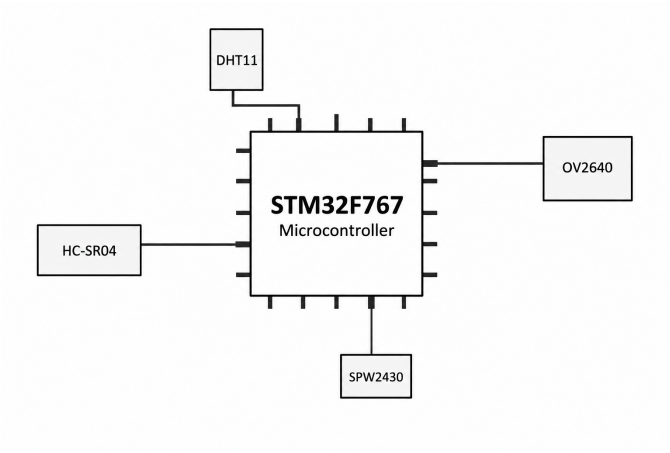


Fig. 1. Sensor platform based on STM32F767.

A. Programming principles

The firmware is written in C using STM32CubeIDE. STM32CubeMX generates peripheral initialization code, while the hardware abstraction layer (HAL) provides access to the peripherals. Application code is placed in `USER CODE` regions. Fig. 2 shows an initialization example.

Macros in `app_config.h` configure the scheduler, chunk size, and enabled tasks. The example of such macros are provided in Table II. Synthetic tasks generate a controlled processor workload without accessing sensors, allowing scheduling behavior to be evaluated under different loads.

```
862   /* Configure the system clock */
863   SystemClock_Config();
864
865   /* USER CODE BEGIN SysInit */
866
867   PlatformTime_Init(HAL_RCC_GetHCLKFreq() / 1000000U);
868
869   /* USER CODE END SysInit */
870
871   /* Initialize all configured peripherals */
872   MX_GPIO_Init();
873   MX_ETH_Init();
874   MX_USART3_UART_Init();
875   MX_USB_OTG_FS_PCD_Init();
876   MX_TIM3_Init();
```

Fig. 2. Peripheral initialization and a user-code region in STM32CubeIDE.

TABLE II
MAIN FIRMWARE SETTINGS.

Macro	Purpose
SCHED_ALGO	Select SuperLoop or chunked EDF
EDF_CHUNK_US	Set the maximum chunk size in microseconds
ENABLE_SYNTHETIC	Enable synthetic tasks
ENABLE_REAL_SENSORS	Enable physical-sensor tasks

III. System Model

We use the periodic task model of Liu and Layland [5] and adapt the task and release notation of Burmyakov and Bini [10]. Table III summarizes the notation.

A real-time system $\mathcal{T} = \{\tau_1, \ldots, \tau_n\}$ consists of n periodic tasks executed on one core ($m = 1$). Each task $\tau_i = (C_i, D_i, P_i)$ is defined by the worst-case execution time C_i of a job, a relative deadline D_i, and a period P_i. We consider implicit deadlines, $D_i = P_i$.

Let $J_{i,j}$ denote the j-th job of τ_i, released at $r_{i,j}$. Consecutive jobs are released one period apart:

$$r_{i,j+1} = r_{i,j} + P_i. \tag{1}$$

Each job $J_{i,j}$ has an absolute deadline $d_{i,j} = r_{i,j} + D_i$. If completed, its finish time is denoted by $f_{i,j}$.

TABLE III
SYSTEM MODEL NOTATION.

Symbol	Meaning
$\mathcal{T}, n$	Task set and number of tasks
τ_i	Task i
C_i	Worst-case execution time of a job of τ_i
P_i	Task period
$D_i = P_i$	Relative deadline, equal to the period
$J_{i,j}$	The j-th job of task τ_i
$r_{i,j}$	Release time of job $J_{i,j}$
$f_{i,j}$	Job completion time
$d_{i,j} = r_{i,j} + D_i$	Absolute deadline of job $J_{i,j}$
m	Number of cores
$U_i = C_i/P_i$	Task utilization
$U_{\mathcal{T}} = \sum_{\tau_i \in \mathcal{T}} U_i$	Task-set utilization

- A schedule specifies which job executes at each time, if any.
- Preemption interrupts a job before completion to execute another job. The interrupted job may resume later.
- A completed job $J_{i,j}$ misses its deadline if $f_{i,j} > d_{i,j}$.
- The response time of a completed job $J_{i,j}$ is $f_{i,j} - r_{i,j}$.

IV. Schedulers

We consider two schedulers: SuperLoop and chunked Earliest Deadline First (EDF). For illustration, we also show a fully preemptive EDF scheduler.

A. SuperLoop

SuperLoop repeatedly polls tasks in a fixed order. For each task, it checks whether the next release time has been reached. If so, the job runs to completion before the next task is polled. The key steps are outlined in Algorithm 1.

Algorithm 1 SuperLoop Scheduler

```
function SUPERLOOP(T)
    t_i^next ← 0,  i = 1, ..., n                ▷ Initial releases
    while true do
        for i ← 1 to n do                        ▷ Fixed task order
            t^cur ← NOW()                        ▷ Read current system time
            if t_i^next ≤ t^cur then
                J_{i,j} ← RELEASEJOB(τ_i, t_i^next)
                EXECUTE(J_{i,j})                 ▷ Non-preemptive
                f_{i,j} ← NOW()
                t_i^next ← t_i^next + P_i
                while t_i^next < f_{i,j} do t_i^next ← t_i^next + P_i
            end if
        end for
    end while
end function
```

Fig. 3(a) illustrates the SuperLoop schedule for tasks τ_1 and τ_2. Both tasks release their first jobs at $t = 0$. Since τ_1 comes first in the polling order, $J_{1,1}$ executes until $t = 18$ ms. Job $J_{2,1}$ then starts and completes at $f_{2,1} = 28$ ms, missing its deadline $d_{2,1} = 20$ ms.

The release at $r_{2,2} = 20$ ms occurs while $J_{2,1}$ is still running. Once this job completes, the scheduler counts that release as skipped and advances the next release time to $r_{2,3} = 40$ ms. Polling continues with no ready job until then.

B. Earliest Deadline First

In contrast, fully preemptive EDF selects the ready job with the earliest absolute deadline. A newly released job preempts the running job if its deadline is earlier.

Fig. 3(b) shows the EDF schedule for the same task set. At $t = 0$, $J_{2,1}$ has the earlier deadline and executes until $t = 10$ ms. Job $J_{1,1}$ then starts, but is preempted at $r_{2,2} = 20$ ms because the new job has a deadline $d_{2,2} = 40$ ms, earlier than $d_{1,1} = 50$ ms. Job $J_{2,2}$ executes until $t = 30$ ms, after which $J_{1,1}$ resumes and completes at $t = 38$ ms. The next job of τ_2 executes from $t = 40$ to $t = 50$ ms. Thus, no deadline is missed and no release is skipped in this example.

C. Chunked Earliest Deadline First

Unlike classical EDF, our chunked EDF implementation executes jobs in non-preemptive chunks of at most Δ ms. Sensor operations are split only at points that preserve the timing requirements of the interface.

Algorithm 2 outlines chunked EDF. The scheduler maintains a set $\mathcal{R}$ of released but unfinished jobs. After each chunk, it updates $\mathcal{R}$ with newly released jobs and selects the job with the earliest absolute deadline.

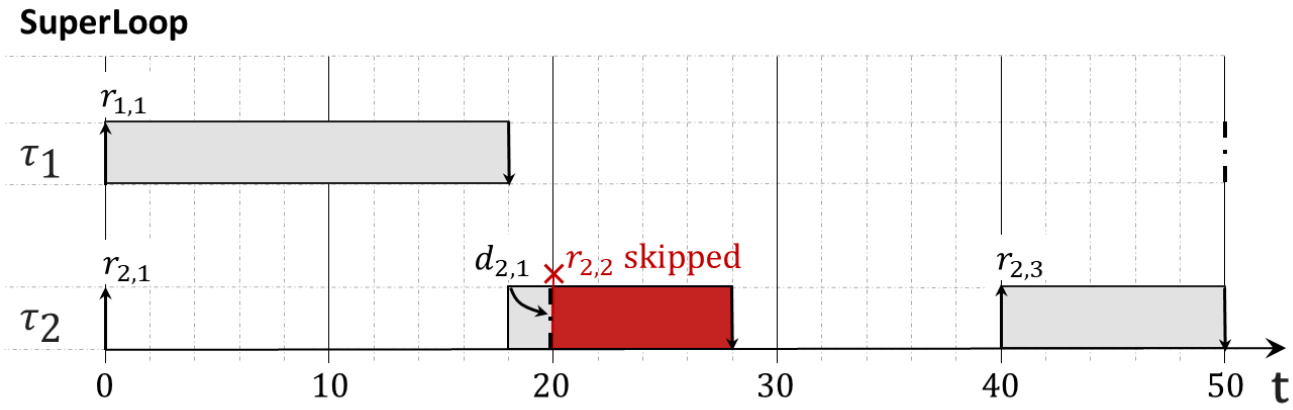
Fig. 3(c) shows the same task set with $\Delta = 4$ ms. At $r_{2,2} = 20$ ms, the first job of τ_1 is still executing a chunk, which ends at $t = 22$ ms. The new job of τ_2 therefore starts at $t = 22$ ms, with a preemption delay of $\delta = 2$ ms. It executes in chunks of 4, 4, and 2 ms and completes at $t = 32$ ms. The shorter final chunk reflects the remaining work. The interrupted job of τ_1 then resumes and completes at $t = 38$ ms.

Algorithm 2 Chunked EDF Scheduler

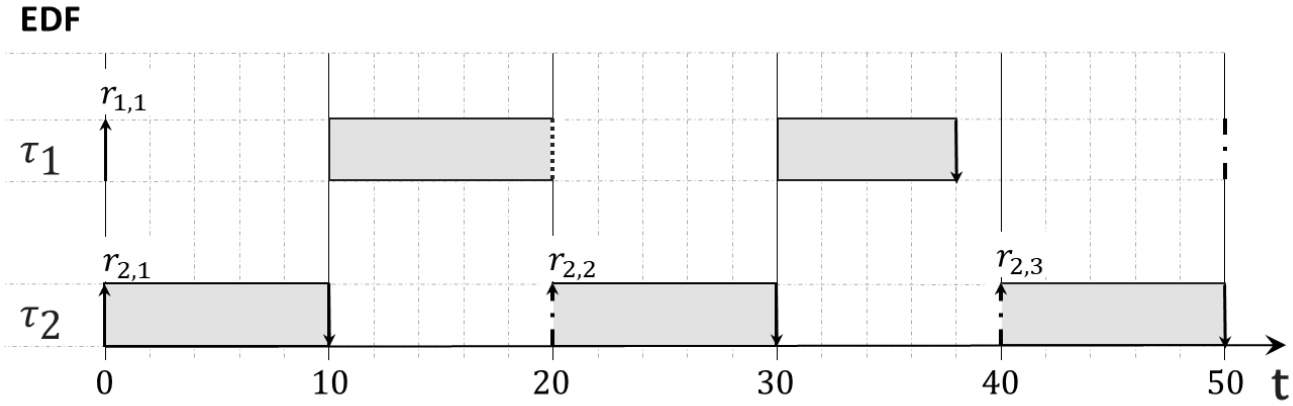
function CHUNKEDEDF($\mathcal{T}, \Delta$)
  $\mathcal{R} \leftarrow \emptyset$ ▷ Set of unfinished jobs
  $t_i^{\mathsf{next}} \leftarrow 0, \quad i = 1, \ldots, n$ ▷ Initial releases
  while true **do**
    $t^{\mathsf{cur}} \leftarrow$ NOW() ▷ Read current system time
    for all $\tau_i \in \mathcal{T}$ **do** ▷ Check releases
      if $t_i^{\mathsf{next}} \leq t^{\mathsf{cur}}$ and no τ_i job in $\mathcal{R}$ **then**
        $J_{i,j} \leftarrow$ RELEASEJOB($\tau_i, t_i^{\mathsf{next}}$)
        $\mathcal{R} \leftarrow \mathcal{R} \cup \{J_{i,j}\}$ ▷ Add new job
      end if
    end for
    if $\mathcal{R} \neq \emptyset$ **then** ▷ Pending work
      $J_{i,j} \leftarrow$ SELECTEDF($\mathcal{R}$) ▷ Earliest deadline
      EXECUTECHUNK($J_{i,j}, \Delta$) ▷ Duration $\leq \Delta$
      if COMPLETED($J_{i,j}$) **then**
        $f_{i,j} \leftarrow$ NOW()
        while $t_i^{\mathrm{next}} < f_{i,j}$ **do** $t_i^{\mathrm{next}} \leftarrow t_i^{\mathrm{next}} + P_i$
        $\mathcal{R} \leftarrow \mathcal{R} \setminus \{J_{i,j}\}$ ▷ Remove completed job
      end if
    end if
  end while
end function

$$\tau_i = (C_i, D_i, P_i), \qquad D_i = P_i, \qquad m = 1$$
$$\tau_1 = (18, 50, 50), \qquad \tau_2 = (10, 20, 20)$$

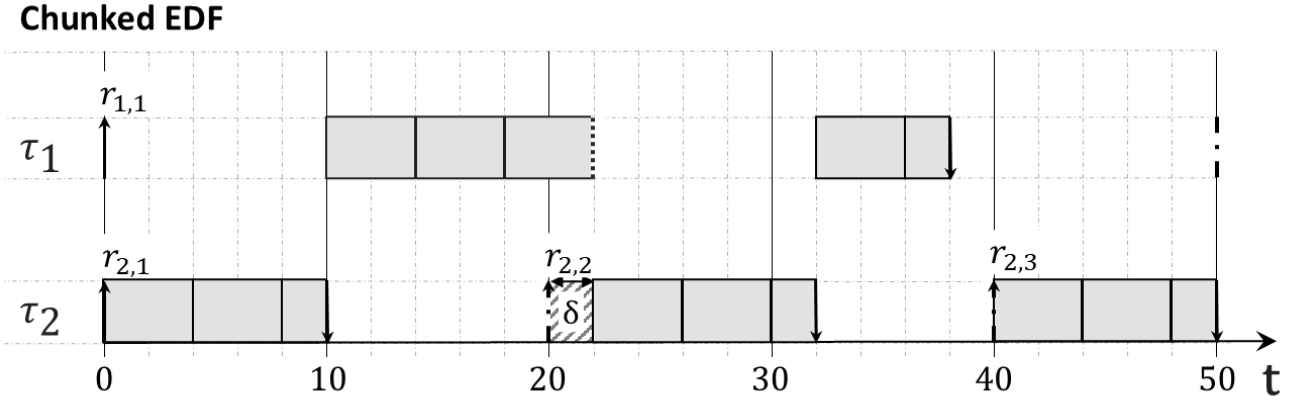
All times in ms. Maximum chunk duration in (c): $\Delta = 4$ ms.



(a) SuperLoop: missed deadline and skipped job release.



(b) Preemptive EDF: immediate preemption by a newly released job with an earlier deadline.



(c) Chunked EDF: delayed preemption until the current chunk completes.



Fig. 3. Comparison of SuperLoop, preemptive EDF, and chunked EDF schedules for the same periodic task set.

REFERENCES

[1] *STM32F765xx, STM32F767xx, STM32F768Ax, STM32F769xx: Datasheet*, STMicroelectronics. [Online]. Available: https://www.st.com/resource/en/datasheet/stm32f767zi.pdf

[2] STMicroelectronics, "Process and environment monitoring," accessed: Oct. 6, 2026. [Online]. Available: https://www.st.com/en/applications/factory-automation/process-and-environment-monitoring.html

[3] ——, "Condition monitoring / predictive maintenance," accessed: Oct. 6, 2026. [Online]. Available: https://www.st.com/en/applications/factory-automation/condition-monitoring-predictive-maintenance.html

[4] Arm Keil, "Execution models: Single task system (Superloop)," accessed: Oct. 4, 2026. [Online]. Available: https://www.keil.com/pack/doc/mw/Graphic/html/gui_exec_model.html

[5] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, Jan. 1973.

[6] *DHT11 Humidity & Temperature Sensor: Technical Data Sheet*, Aosong Electronics. [Online]. Available: https://www.mouser.com/datasheet/2/758/DHT11-Technical-Data-Sheet-Translated-Version-1143054.pdf

[7] *Ultrasonic Ranging Module HC-SR04*, Elecfreaks. [Online]. Available: https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf

[8] *OV2640 Datasheet*, OmniVision Technologies. [Online]. Available: https://www.uctronics.com/download/OV2640_DS.pdf

[9] *SPW2430HR5H-B: Top Port SiSonic Microphone*, Knowles Electronics. [Online]. Available: https://www.mouser.com/datasheet/2/218/SPW2430HR5H-B-1290924.pdf

[10] A. Burmyakov and E. Bini, "Improving the efficiency of global EDF exact schedulability test," *IEEE Transactions on Computers*, pp. 1–14, 2026.